

Implementation of RPY hydrodynamics with rotations

January 22, 2026

1 RPY hydrodynamics implementation tests

1.0.1 Two spheres mobilities

- From [2006-reichert-phdth], Section 2.4.5.

Two spheres of radius $a = 1$ placed in the z direction with center to center distances r (in units of a). The fluid viscosity is set to $\eta = 1$ for simplicity (corresponding approximately to water).

```
[1]: open("tensor_analysis_file.csv", "w").close()

import espressomd
import numpy as np
import json
import os

def load_diffusion_tensors(filename="tensor_analysis_file.csv"):
    tensors = []
    current = []

    with open(filename) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            if line.startswith("#"):
                if current:
                    tensors.append(np.array(current, dtype=float))
                    current = []
            else:
                current.append([float(x) for x in line.split(",")])

        if current:
            tensors.append(np.array(current, dtype=float))

    return tensors

system = espressomd.System(box_l = [10.0, 10.0, 10.0])
```

```

system.time_step = 0.000001
system.cell_system.skin = 0.1

"""Setup for Brownian mechanics"""

system.thermostat.set_brownian(kT = 0.0, gamma = 6.0, gamma_rotation = 8.0,
↪seed = 1)
system.integrator.set_brownian_dynamics(2.0)

rs = [2.0 + 0.1 * i for i in range(int((5.0 - 2.0) / 0.1) + 1)]

for r in rs:

    system.part.add(pos = [0.0, 0.0, 0.0])
    system.part.add(pos = [0.0, 0.0, r])
    system.integrator.run(1)
    system.part.clear()

tensors = load_diffusion_tensors()

mtt12zz = []
mtt12xx = []
mrr12zz = []
mrr12xx = []
mtr12xy = []
mtt11xx = []
mtt11zz = []
mrr11xx = []
mrr11zz = []
mtr11xy = []

for M in tensors:

    mtt12xx.append(M[0, 3])
    mtt12zz.append(M[2, 5])
    mrr12zz.append(M[6, 9])
    mrr12xx.append(M[8, 11])
    mtr12xy.append(M[0, 10])
    mtt11xx.append(M[0, 0])
    mtt11zz.append(M[2, 2])
    mrr11xx.append(M[6, 6])
    mrr11zz.append(M[8, 8])
    mtr11xy.append(M[0, 7])

import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 3, figsize=(12, 10), sharex=False)

```

```

ax = axes[0, 0]
ax.plot(rs, mtt11xx, label=r"$\perp$", linestyle="--", marker="o")
ax.plot(rs, mtt11zz, label=r"$\parallel$")
ax.legend()
ax.text(0.90, 0.05, "(a)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4a")
ax.set_ylim(0.13, 0.17)
ax.set_xlim(2., 5.)
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{11}^{\text{tt}}$ [1/($\pi$ $\eta$ a$)]", fontsize=13)
ax.set_yticks([0.13, 0.14, 0.15, 0.16, 0.17])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax2 = ax.twinx()
ax2.set_ylim(0.13, 0.17)
ax2.set_yticks([0.13, 0.14, 0.15, 0.16, 0.17])

ax = axes[0, 1]
ax.plot(rs, mrr11xx, label=r"$\perp$", linestyle="--", marker="o")
ax.plot(rs, mrr11zz, label=r"$\parallel$")
ax.legend()
ax.text(0.90, 0.05, "(c)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4c")
ax.set_ylim(0.09, 0.13)
ax.set_xlim(2., 5.)
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{11}^{\text{rr}}$ [1/($\pi$ $\eta$ a3)]", fontsize=13)
ax.set_yticks([0.09, 0.10, 0.11, 0.12, 0.13])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax2 = ax.twinx()
ax2.set_ylim(0.09, 0.13)
ax2.set_yticks([0.09, 0.10, 0.11, 0.12, 0.13])

ax = axes[0, 2]
ax.plot(rs, mtr11xy, label=r"$\perp$")
ax.legend()
ax.text(0.90, 0.05, "(e)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4e")
ax.set_ylim(-0.004, 0.012)
ax.set_xlim(2., 5.)
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{11}^{\text{tr}}$ [1/($\pi$ $\eta$ a2)]", fontsize=13)
ax.set_yticks([-0.004, 0.000, 0.004, 0.008, 0.012])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax2 = ax.twinx()
ax2.set_ylim(-0.004, 0.012)
ax2.set_yticks([-0.004, 0.000, 0.004, 0.008, 0.012])

```

```

ax = axes[1, 0]
ax.plot(rs, mtt12xx, label=r"$\perp$")
ax.plot(rs, mtt12zz, label=r"$\parallel$")
ax.text(0.05, 0.90, "(b)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4b")
ax.set_ylim(0.02, 0.14)
ax.set_xlim(2., 5.)
ax.set_yticks([0.02, 0.05, 0.08, 0.11, 0.14])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax.legend()
ax2 = ax.twinx()
ax2.set_ylim(0.02, .14)
ax2.set_yticks([0.02, 0.05, 0.08, 0.11, 0.14])
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{12}^{\tt t}$ $[1/(\pi \eta a)]$", fontsize=13)

ax = axes[1, 1]
ax.plot(rs, mrr12xx, label=r"$\perp$")
ax.plot(rs, mrr12zz, label=r"$\parallel$")
ax.legend()
ax.text(0.05, 0.90, "(d)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4d")
ax.set_ylim(-0.02, 0.02)
ax.set_xlim(2., 5.)
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{12}^{\tt rr}$ $[1/(\pi \eta a^3)]$", fontsize=13)
ax.set_yticks([-0.02, -0.01, 0., 0.01, 0.02])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax2 = ax.twinx()
ax2.set_ylim(-0.02, 0.02)
ax2.set_yticks([-0.02, -0.01, 0.00, 0.01, 0.02])

ax = axes[1, 2]
ax.plot(rs, mtr12xy, label=r"$\perp$")
ax.legend()
ax.text(0.05, 0.90, "(f)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.set_title("Compare to fig. 2.4f")
ax.set_ylim(0., 0.04)
ax.set_xlim(2., 5.)
ax.set_xlabel("$r/a$", fontsize=13)
ax.set_ylabel(r"$\mu_{12}^{\tt tr}$ $[1/(\pi \eta a^2)]$", fontsize=13)
ax.set_yticks([0., 0.01, 0.02, 0.03, 0.04])
ax.set_xticks([2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
ax2 = ax.twinx()

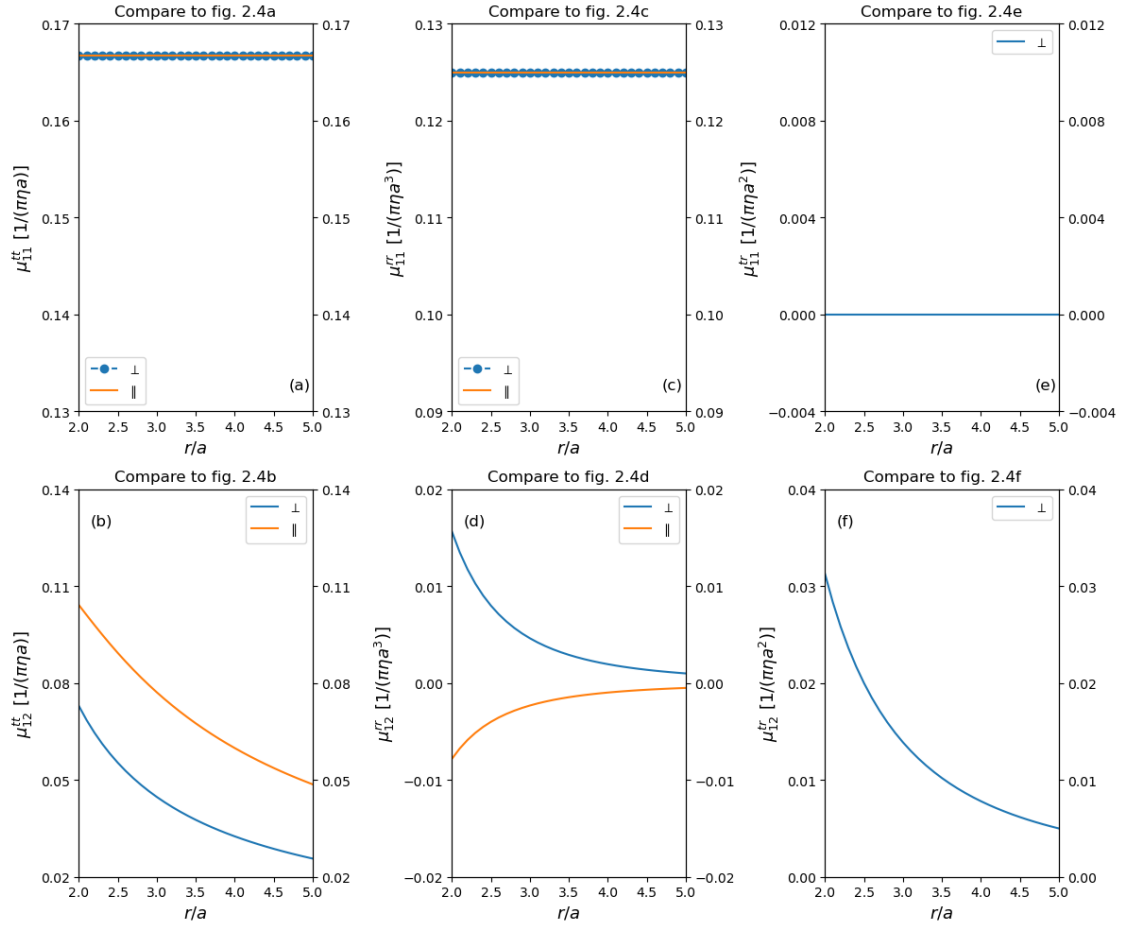
```

```

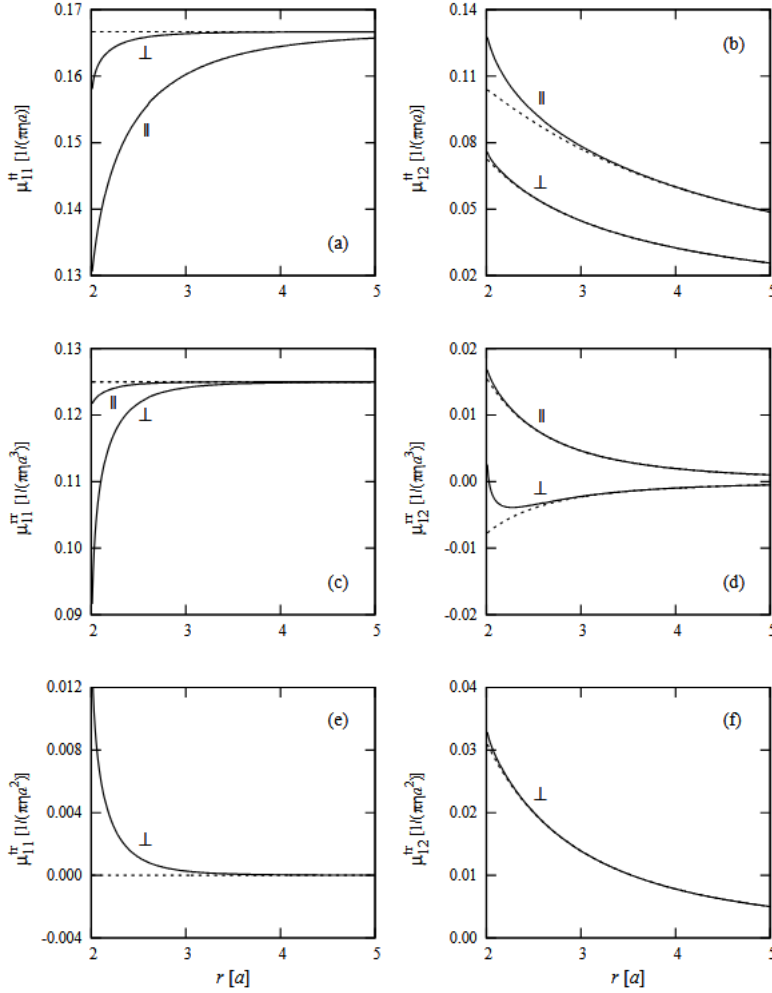
ax2.set_ylim(0., 0.04)
ax2.set_yticks([0., 0.01, 0.02, 0.03, 0.04])

plt.tight_layout()
plt.savefig("Tensor_analysis_all_in_one.png")

```



Corresponding plot taken from: M. Reichert: Hydrodynamic Interactions in Colloidal and Biological Systems. Section 2.4.5



Equilateral triangle, perpendicular forces

- Ref. [2013-wilson-jcmp], extended by [2025-torres-arxiv]

Three spheres of radius $a = 1$ placed in an equilateral triangle in the x-y plane with center to center distances r (in units of a). A constant Force $F = (0, 0, f)$ in z direction, but no torque, is applied on each particle. The fluid viscosity is again set to $\eta = 1$ for simplicity (corresponding approximately to water).

```
[1]: open("velocity_analysis_file.csv", "w").close()

import espressomd
import numpy as np
import json
import os
import matplotlib.pyplot as plt

def load_vectors(filename="velocity_analysis_file.csv"):
    vectors = []
```

```

with open(filename) as f:
    for line in f:
        line = line.strip()

        if not line or line.startswith("#"):
            continue

        vec = np.array([float(x) for x in line.split(",")])
        vectors.append(vec)

    return vectors

# tabulated data from [2014-wilson-jcp]
sWs = np.array([2.01, 2.05, 2.1, 2.15, 2.2, 2.25, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8,
↪ 2.9, 3.0])
vWs = np.array([
1.79223228,
1.79070892,
1.77826951,
1.76311004,
1.74703222,
1.73072508,
1.71452988,
1.68314045,
1.65356957,
1.62599574,
1.6003915,
1.57664602,
1.55461831,
1.53416156
])
wWs = np.array([
0.15960749,
0.195478184,
0.209731929,
0.214756269,
0.215412007,
0.213577673,
0.210224103,
0.201041187,
0.190411103,
0.179481457,
0.16880121,
0.158630515,
0.149081378,
0.140187608

```

```

])

system = espressomd.System(box_l = [10.0, 10.0, 10.0])
system.time_step = 0.000001
system.cell_system.skin = 1
system.thermostat.set_brownian(kT = 0.0, gamma = 6.0 * np.pi, gamma_rotation = 8.0 * np.pi, seed = 1)
system.integrator.set_brownian_dynamics(2.0)

rs = [2.0 + 0.1 * i for i in range(11)]

for r in rs:

    part1 = system.part.add(pos = [0.0, 0.0, 0.0], ext_force = [0.0, 0.0, 1.0])
    part2 = system.part.add(pos = [r, 0.0, 0.0], ext_force = [0.0, 0.0, 1.0])
    part3 = system.part.add(pos = [r/2, (np.sqrt(3)/2) * r, 0], ext_force = [0.0, 0.0, 1.0])

    system.integrator.run(1)
    system.part.clear()

vectors = load_vectors()

V0 = 1.0/(6.0 * np.pi)
W0 = V0

VVOs = [(v[2] / V0) for v in vectors]
WWOs = [(np.linalg.norm(v[-3]) / W0) for v in vectors]

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharex=True)

ax = axes[0]
ax.plot(sWs, vWs, "o:", label = "Wilson")
ax.plot(rs, VVOs, label = "RPY")
ax.set_title("Compare to fig. 1a in [2025-torres-arxiv]")
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel("$V_z / V_0$", fontsize=15)
ax.text(0.90, 0.75, "(a)", transform=ax.transAxes, fontsize=12, va="top")
ax.legend()

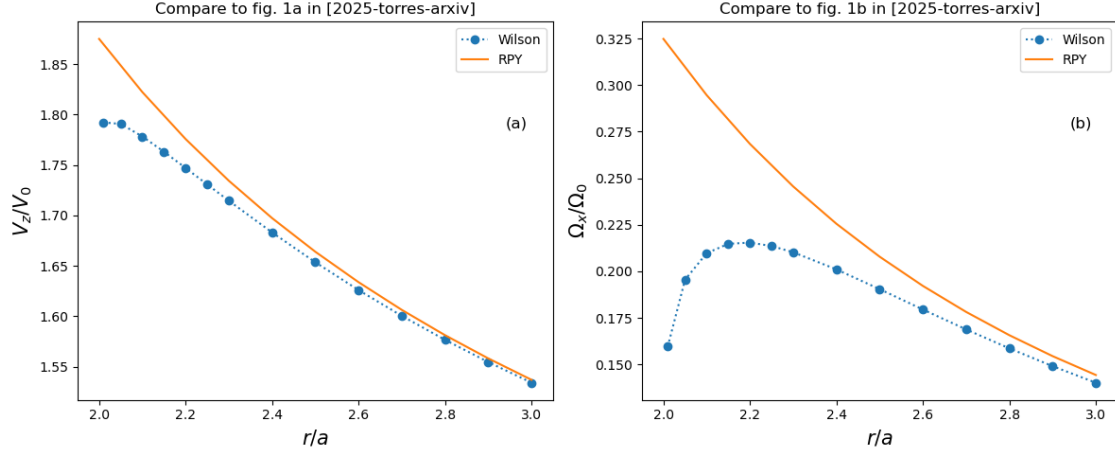
ax = axes[1]
ax.plot(sWs, wWs, ":o", label = "Wilson")
ax.plot(rs, WWOs, label = "RPY")
ax.set_title("Compare to fig. 1b in [2025-torres-arxiv]")
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel("$\Omega_x / \Omega_0$", fontsize=15)
ax.text(0.90, 0.75, "(b)", transform=ax.transAxes, fontsize=12, va="top")

```



```
ax.legend()

plt.tight_layout()
plt.savefig("Velocity_analysis_all_in_one_fz.png")
```

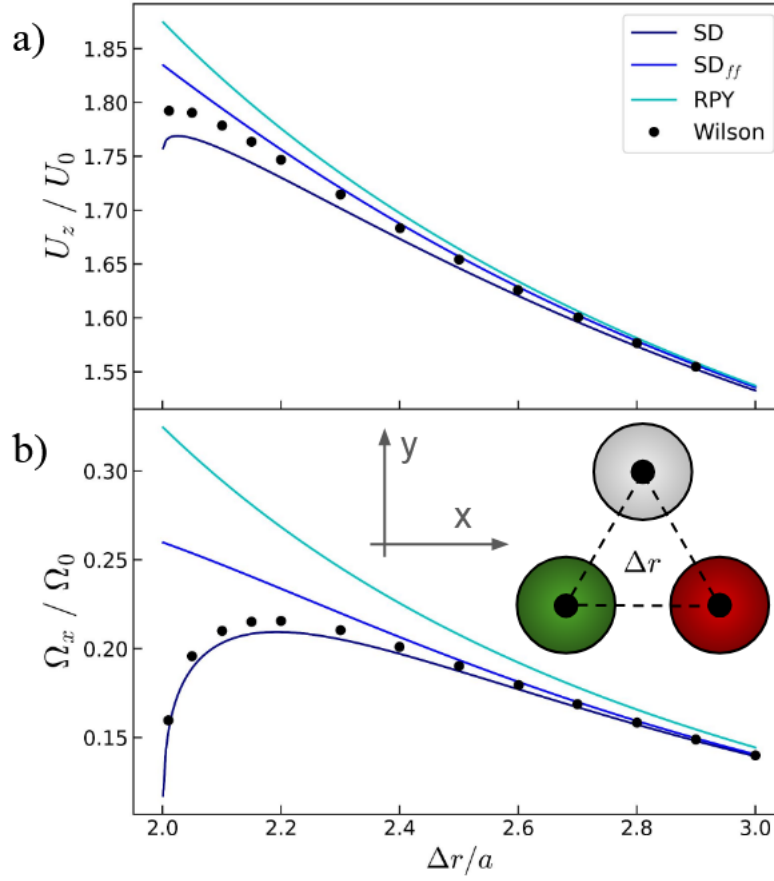


Tabulated data for the same configuration taken from: H. Wilson: Stokes flow past three spheres. Section 3.1

Table 1
Mobility and resistance results for a system of three equidistant spheres moving out of their plane.

s	M_{UF}	M_{DF}	M_{UR}	M_{UR}	R_{FU}
3.00	1.53416156	0.140187608	0.771951396	0.140187608	0.662821
2.90	1.55461831	0.149081378	0.772261325	0.149081378	0.655377
2.80	1.57664602	0.158630515	0.771969753	0.158630515	0.647648
2.70	1.60039150	0.168801210	0.770692987	0.168801210	0.639623
2.60	1.62599574	0.179481457	0.767809702	0.179481457	0.631297
2.50	1.65356957	0.190411103	0.762285310	0.190411103	0.622662
2.40	1.68314045	0.201041187	0.752325593	0.201041187	0.613716
2.30	1.71452988	0.210224103	0.734617762	0.210224103	0.604460
2.25	1.73072508	0.213577673	0.720980855	0.213577673	0.599716
2.20	1.74703222	0.215412007	0.702342997	0.215412007	0.594897
2.15	1.76311004	0.214756269	0.676197489	0.214756269	0.590004
2.10	1.77826951	0.209731929	0.637667933	0.209731929	0.585039
2.05	1.79070892	0.195478184	0.573859459	0.195478184	0.580005
2.01	1.79223228	0.159607490	0.455596646	0.159607101	0.575932

Corresponding plot taken from: K. W. Torre, J. d. Graaf.: Hydrodynamic Interactions in Particle Suspensions: A Perspective on Stokesian Dynamics. Section 4 A.



Equilateral triangle, one particle with an in-plane orientation

- Ref. [2013-wilson-jcmp], extended by [2025-torres-arxiv]

Equilateral triangle configuration, with in-plane force applied to a single particle, pushing it towards the center of the triangle.

```
[1]: open("velocity_analysis_file.csv", "w").close()

import espressomd
import espressomd.io.writer.vtf
import numpy as np
import json
import os
import matplotlib.pyplot as plt

def load_vectors(filename="velocity_analysis_file.csv"):
    vectors = []

    with open(filename) as f:
        for line in f:
            line = line.strip()
```

```

        if not line or line.startswith("#"):
            continue

        vec = np.array([float(x) for x in line.split(",")])
        vectors.append(vec)

    return vectors

# tabulated data from [2014-wilson-jcp]
sWs = np.array([2.01, 2.10, 2.5, 3.0, 4.0, 6.0])
vWs = np.array([
    0.65528,
    0.73857,
    0.87765,
    0.93905,
    0.97964,
    0.99581,
])
wWs = np.array([
    0.037336,
    0.052035,
    0.045466,
    0.035022,
    0.021634,
    0.010159,
])
vWs2 = np.array([
    0.63461,
    0.59718,
    0.49545,
    0.41694,
    0.31859,
    0.21586,
])
vWs3 = np.array([
    0.00498,
    0.03517,
    0.07393,
    0.07824,
    0.06925,
    0.05078,
])

system = espressomd.System(box_l = [20.0, 20.0, 20.0])
system.time_step = 0.000001
system.cell_system.skin = 1

```

```

system.thermostat.set_brownian(kT = 0.0, gamma = 6.0 * np.pi, gamma_rotation = 8.0 * np.pi, seed = 1)
system.integrator.set_brownian_dynamics(2.0)

rs = [2.0, 2.01, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 5.0, 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6.0]

for r in rs:

    #part1 = system.part.add(pos = [0.0, 0.0, 0.0], ext_force=[6*np.pi * (np.sqrt(3)/2), 6*np.pi * 0.5, 0.0])
    part1 = system.part.add(pos = [0.0, 0.0, 0.0])
    part2 = system.part.add(pos = [r, 0.0, 0.0])
    part3 = system.part.add(pos = [r/2, (np.sqrt(3)/2)*r, 0.0], ext_force=[0.0, -6*np.pi, 0.0])

    system.integrator.run(1)
    system.part.clear()

vectors = load_vectors()
F = 6.0*np.pi
V0 = F/(6.0 * np.pi)
W0 = V0

VV0s = [abs(v[7]) / V0 for v in vectors]
WW0s = [np.linalg.norm(v[12:15] - v[9:12])/(2*W0) for v in vectors]
#parallel component of no force particles
VV02s = [abs(v[1] + v[4])/(2*V0) for v in vectors]
#orthogonal component of no force particles
VV03s = [(v[3] - v[0])/(2*V0) for v in vectors]

fig, axes = plt.subplots(2, 2, figsize=(10, 10), sharex=False)

ax = axes[0, 0]
ax.plot(sWs, vWs, "o:", label = "Wilson")
ax.plot(rs, VV0s, label = "RPY")
ax.set_title("Compare to fig. 2a in [2025-torres-arxiv]")
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel("$V_x / V_0$", fontsize=15)
ax.text(0.50, 0.05, "(a)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.legend()

ax = axes[0, 1]
ax.plot(sWs, vWs2, "o:", label = "Wilson")
ax.plot(rs, VV02s, label = "RPY")
ax.set_title("Compare to fig. 2b in [2025-torres-arxiv]")

```

```

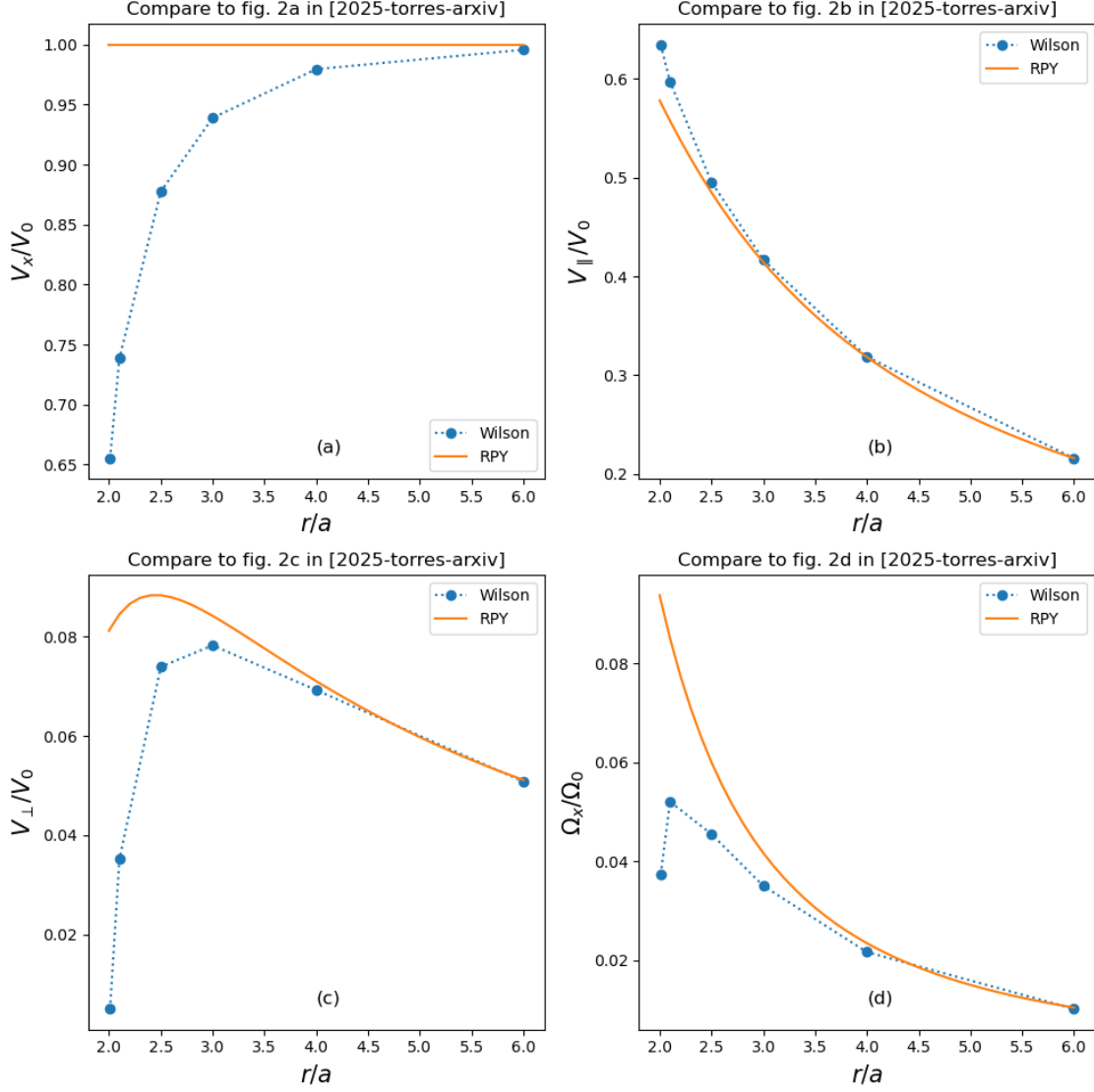
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel(r"$V_{\parallel} / V_0$", fontsize=15)
ax.text(0.50, 0.05, "(b)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.legend()

ax = axes[1, 0]
ax.plot(sWs, vWs3, "o:", label = "Wilson")
ax.plot(rs, VV03s, label = "RPY")
ax.set_title("Compare to fig. 2c in [2025-torres-arxiv]")
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel(r"$V_{\perp} / V_0$", fontsize=15)
ax.text(0.50, 0.05, "(c)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.legend()

ax = axes[1, 1]
ax.plot(sWs, wWs, "o:", label = "Wilson")
ax.plot(rs, WW0s, label = "RPY")
ax.set_title("Compare to fig. 2d in [2025-torres-arxiv]")
ax.set_xlabel("$r/a$", fontsize=15)
ax.set_ylabel(r"$\Omega_x / \Omega_0$", fontsize=15)
ax.text(0.50, 0.05, "(d)", transform=ax.transAxes, fontsize=12, va="bottom")
ax.legend()

plt.tight_layout()
plt.savefig("Velocity_analysis_all_in_one_1force.png")

```



Tabulated data for the same configuration taken from: H. Wilson: Stokes flow past three spheres. Section 4.2

Table 3
Velocities and angular velocities for a triangle of spheres of radius a , arranged in an equilateral triangle of side as , one of which is driven towards the triangle centroid with a force $6\pi\mu a$. It has resultant velocity U_1 ; U_2 is the induced velocity in the other two spheres in the same direction, and $\pm U_3$ their velocity perpendicular to the forcing. They rotate with angular velocity $\pm a\Omega$. Results are shown for our method and for Stokesian Dynamics (SD).

s	Current method				SD			
	U_1	U_2	U_3	Ω	U_1	U_2	U_3	Ω
2.01	0.65528	0.63461	0.00498	0.037336	0.64739	0.62691	0.00451	0.034339
2.10	0.73857	0.59718	0.03517	0.052035	0.73126	0.58784	0.02570	0.051414
2.50	0.87765	0.49545	0.07393	0.045466	0.87482	0.48829	0.05853	0.045446
3.00	0.93905	0.41694	0.07824	0.035022	0.93806	0.41356	0.06970	0.034843
4.00	0.97964	0.31859	0.06925	0.021634	0.97945	0.31774	0.06639	0.021581
6.00	0.99581	0.21586	0.05078	0.010159	0.99579	0.21575	0.05019	0.010153

Corresponding plot taken from: K. W. Torre, J. d. Graaf.: Hydrodynamic Interactions in Particle

